



空间数据库原理与应用

第七讲：空间数据库模型

任课教师：李晓明

建筑与城市规划学院 城市空间信息工程系

其中部分图片来自互联网和同行专家

目录

CONTENTS

01

空间数据库概念及发展

02

空间数据库模型概述

03

集成式的空间数据库模型

目录

CONTENTS

1

空间数据库概念及发展

1.1 空间数据库概念

1.2 空间数据库发展

1.1 空间数据库概念

空间数据库概念

- **空间数据库指的是地理信息系统在计算机物理存储介质上存储的与应用相关的地理空间数据的总和，一般是以一系列特定结构的文件的形式组织在存储介质之上的。**

矢量数据

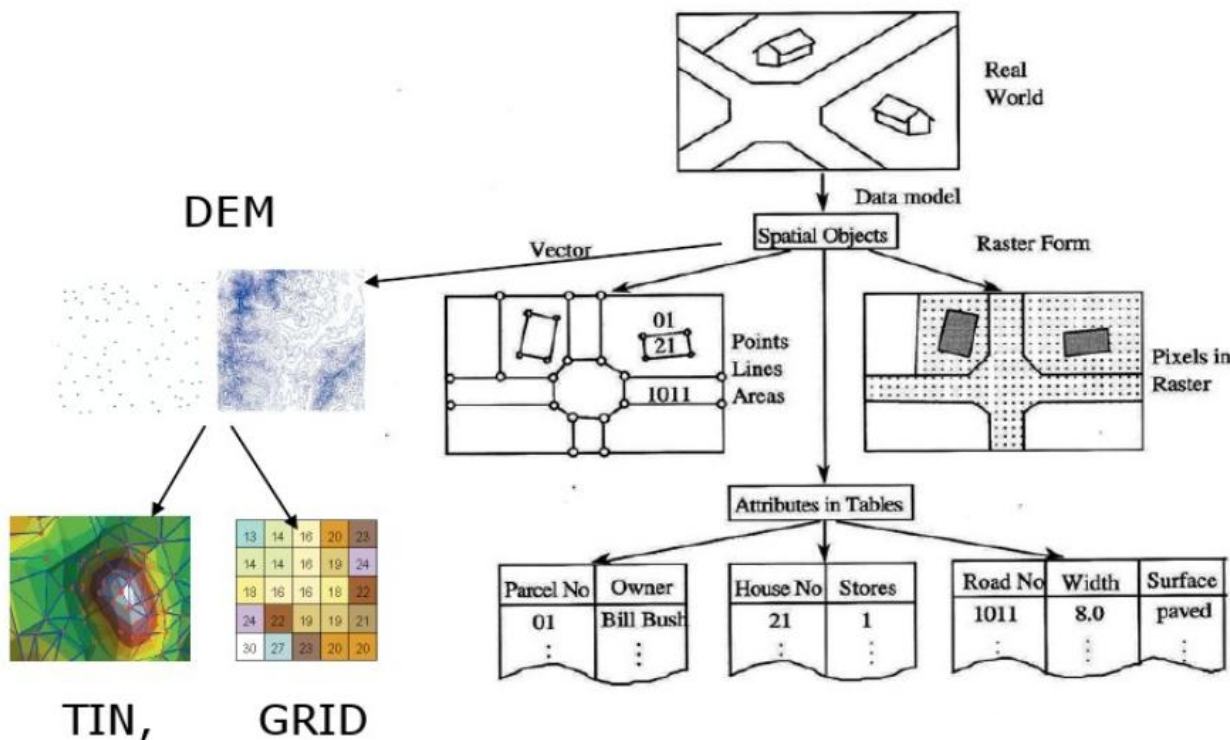
影像数据

地形数据

三维模型数据

元数据

.....



1.1 空间数据库概念

传统数据库与空间数据库的比较

	传统数据库	空间数据库
数据连续性/ 相关性	不连续 相关性小	连续 较强空间相关性
实体类型/ 空间关系	少 简单固定	多 复杂且不固定
记录长度	结构化 等长	非结构化 不等长
查询与操作	文字、数字	文字数字 空间图形

1.1 空间数据库概念

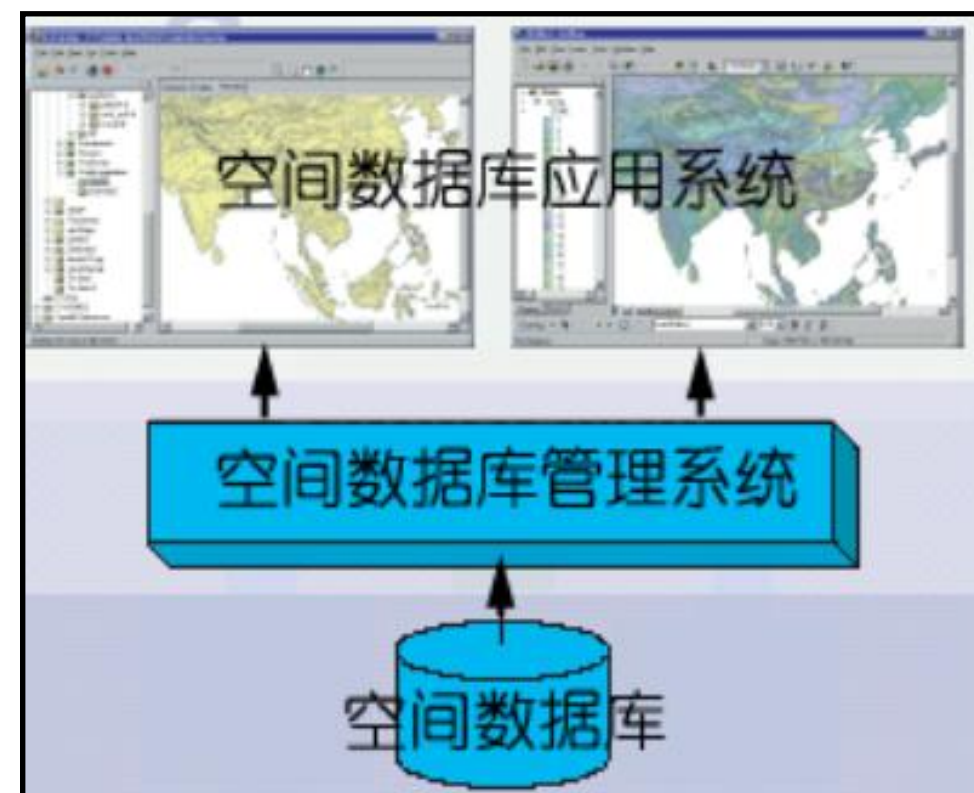
空间数据库系统

➤ **空间数据库系统也是由三部分构成，所不同的是研究对象为空间数据。**

①**空间数据库**：是地理信息系统在计算机物理存储介质上存储的与应用相关的地理空间数据的总和，一般是以一系列特定结构的文件的形式组织在存储介质之上的。

②**空间数据库管理系统**：指能够对物理介质上存储的地理空间数据进行语义和逻辑上的定义，提供必需的空间数据查询和存取功能，以及能够对空间数据进行有效的维护和更新的一套软件系统。

③**空间数据库应用系统**：地理信息系统的应用软件部分，如GIS的空间分析模型和应用模型等。



1.2 空间数据库发展

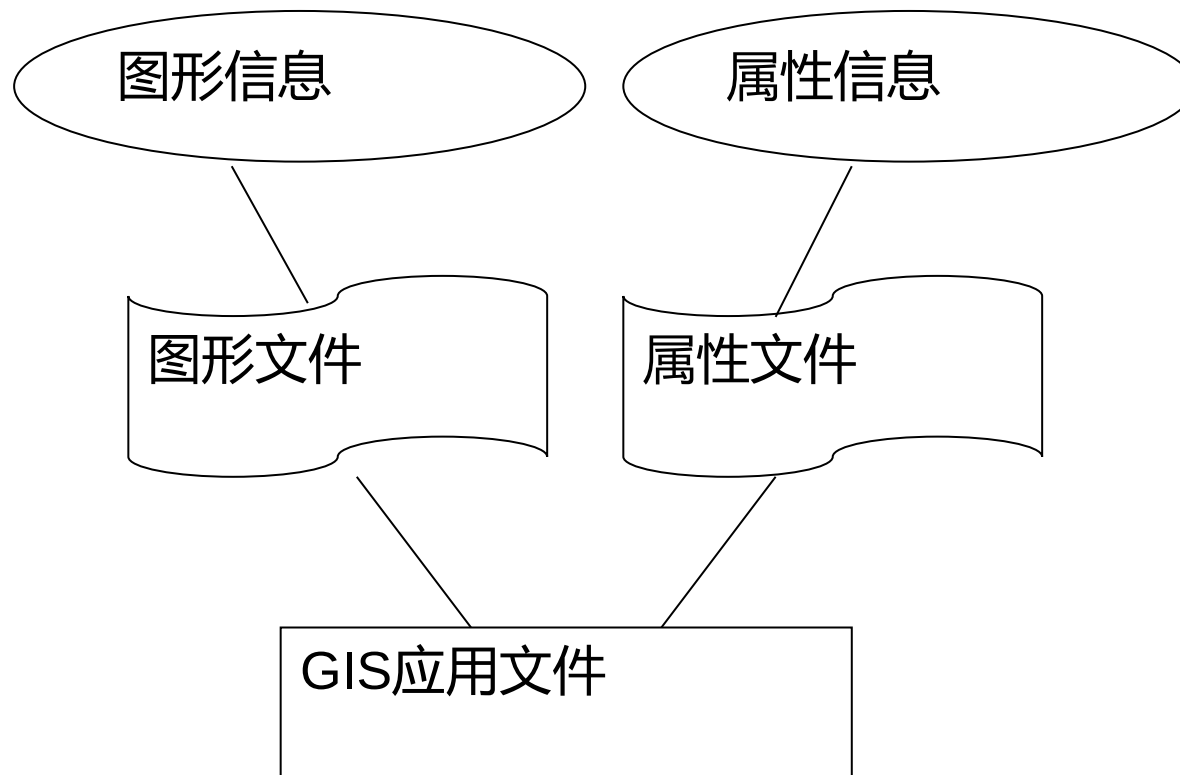
空间数据管理模式发展

- 文件管理模式
- 混合管理模式
- 扩展式管理模式
- 集成式管理模式

2.2 空间数据库发展

1、文件管理模式

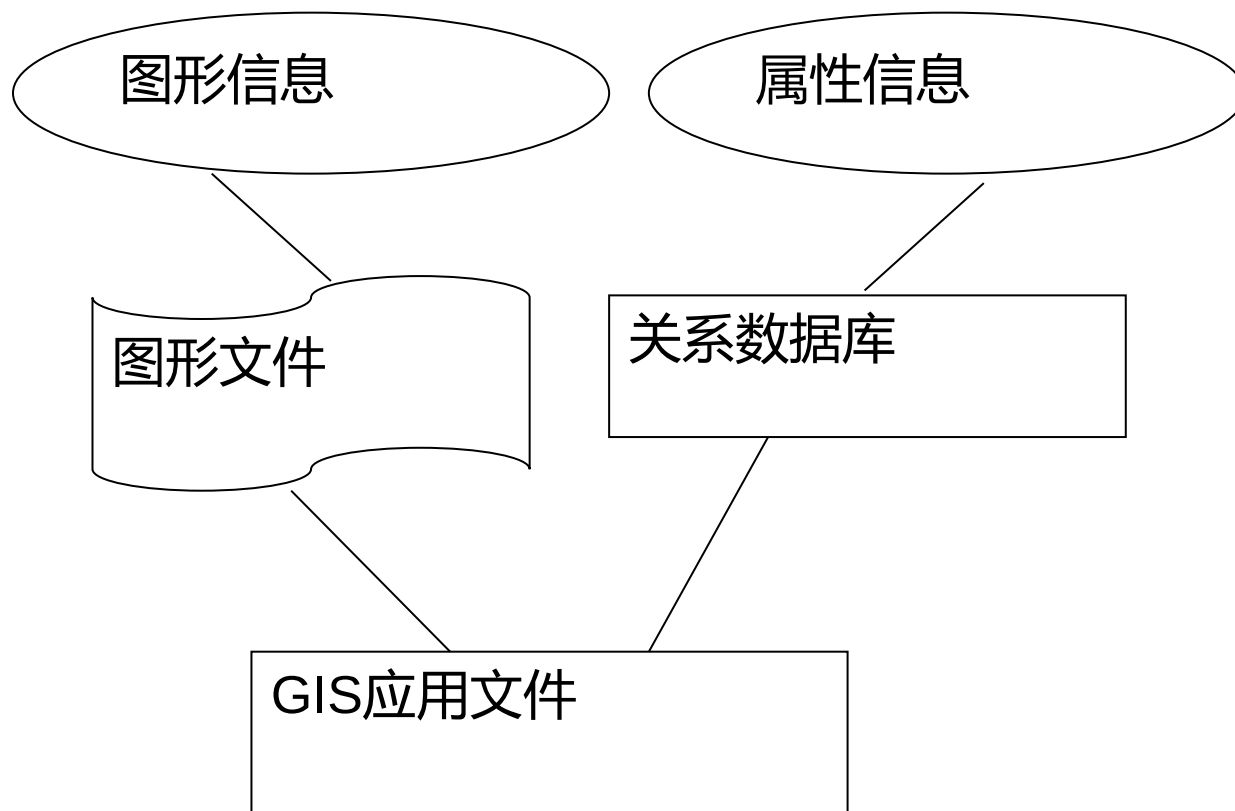
- 最初数据库技术还不是很成熟的时候, GIS应用采用**文件系统**存储数据。
- 每个GIS应用都对应自己的空间和属性数据文件,当多个应用需要访问的数据有相同部分时,就将这些数据提取出来,存放在一个新的公共数据文件中。
- 代表: ArcInfo的Coverage文件管理模式。



2.2 空间数据库发展

2、混合管理模式

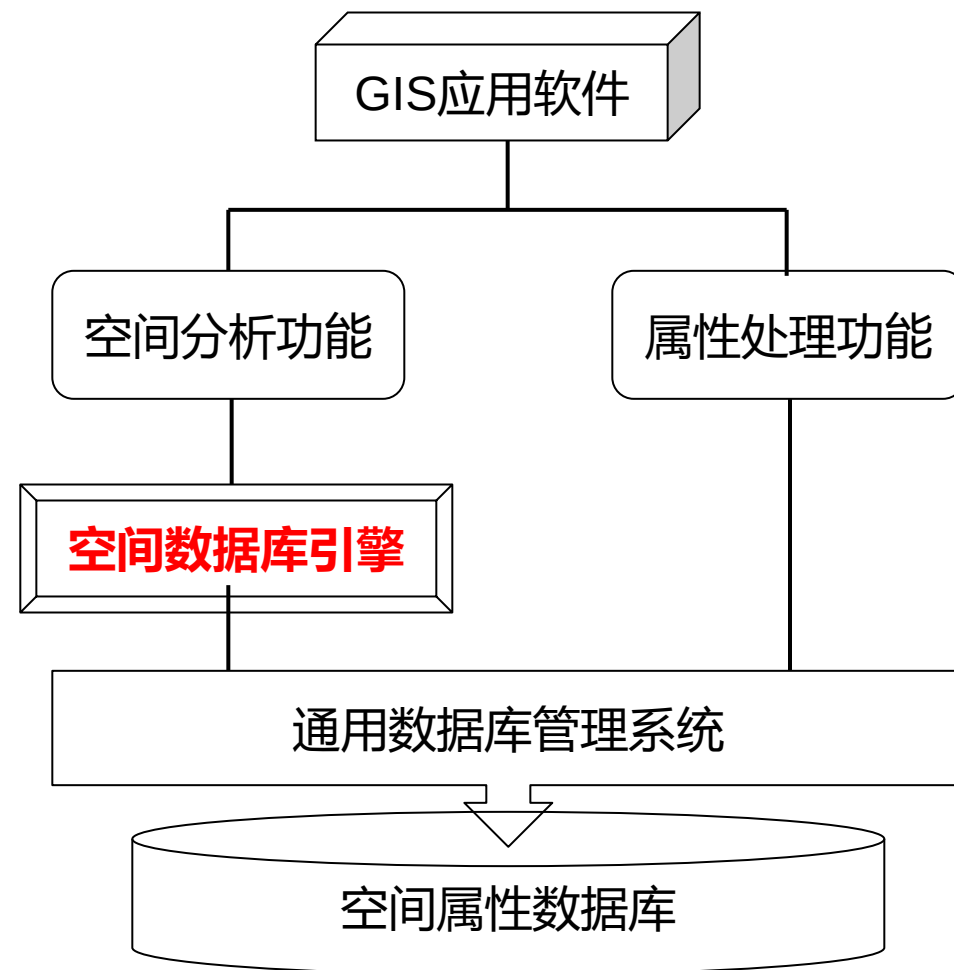
- **定义：**属性数据用关系数据库管理，数据用文件管理，两者间建立联系
- **优点：**实现空间数据管理、速度较快
- **缺点：**数据的安全性、一致性、完整性、并发控制性、数据恢复较差，没有解决空间对象反向检索问题
- **代表：**ArcInfo、ArcView的shape文件和MapInfo的.TAB文件等管理模式



2.2 空间数据库发展

3、扩展式管理模式

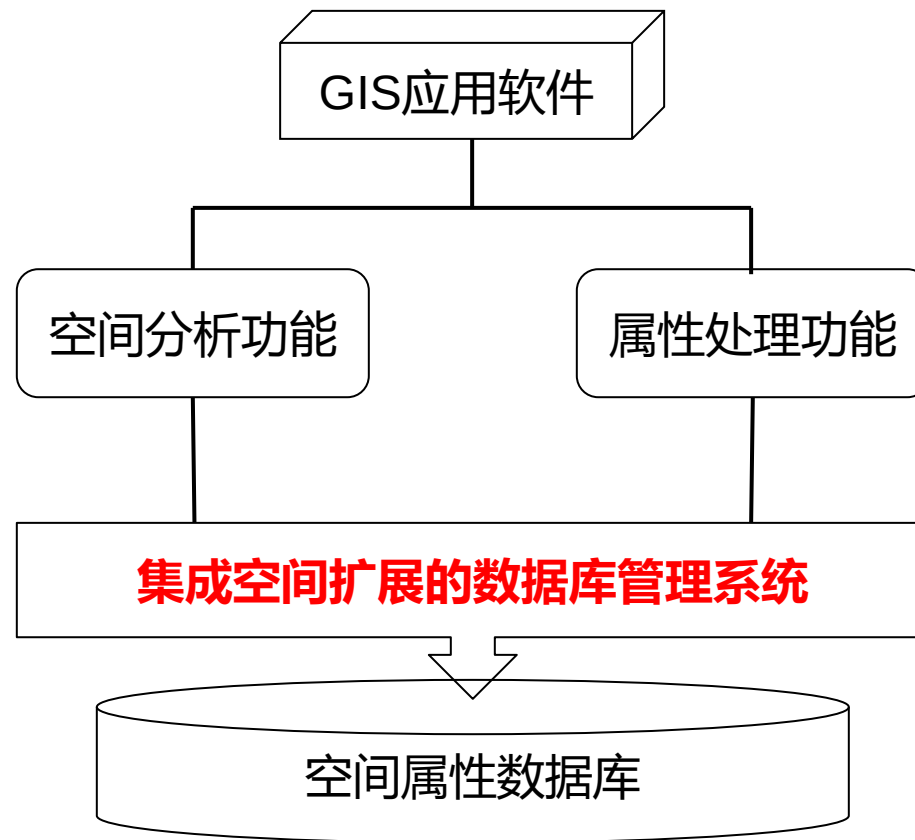
- **定义：**属性数据和图形数据用关系数据库统一管理
- **关系数据库提供了Binay（二进制）字段，变长的坐标数据采用该字段管理**
- **优点：**一体化管理，数据的安全性、一致性、完整性、并发控制性、数据恢复较好
- **缺点：**速度慢，没有解决空间对象反向检索问题
- **代表：**ArcGIS的Geodatabase（ArcSDE）



2.2 空间数据库发展

4、集成式管理模式

- **定义：**关系数据库推出了空间数据管理的模块，定义了操纵空间对象的API函数，属性数据和坐标数据用关系数据库统一管理
- **优点：**一体化管理，数据的安全性、一致性、完整性、并发控制性、数据恢复较好，速度较快
- **缺点：**部分解决空间对象反向检索问题
- **代表：**Oracle Spatial, PostGIS。



目录

CONTENTS

2

空间数据库模型概述

2.1 空间数据库模型概念

2.2 Geodatabase

2.1 空间数据库模型概念

什么是空间数据库模型？

空间数据库模型是将不同类型空间数据的几何、拓扑以及属性等各种信息在数据库管理系统中进行有效存储与管理的数据库模型。

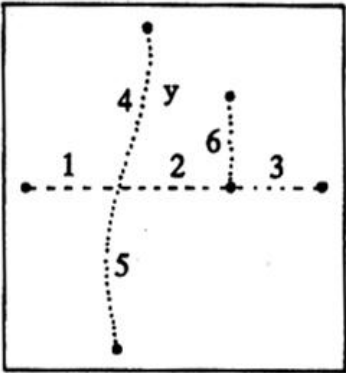
- 以一系列简单但核心的关系数据库概念为基础，并利用了基础数据库管理系统 (DBMS) 的优势
- 简单表和明确定义的属性类型用于存储各地理数据集的方案、规则、库以及空间属性数据
- 可使用结构化查询语言 (SQL) (即一系列关系函数和运算符) 来创建、修改以及查询表及其数据元素

两个核心问题：

- 1、数据库管理系统如何支持空间数据存储？
- 2、如何实现空间数据在数据库管理系统的高效存储与查询检索？

2.1 空间数据库模型概念

道路图:



实体坐标

标识码	x,y 坐标对				
1	1.5	5.5			
2	5.5	8.5			
3	8.5	10.5			
4	5.9	4.8	3.7	4.6	5.5
5	5.5	4.6	3.5	2.4	1.3
6	8.5	8.1			

实体属性

标识号	道路类型	路面材料	宽度	行车道数	道路名称
1	2	柏油	48	4	解放路
2	2	柏油	48	4	珞瑜路
3	2	柏油	48	4	中北路
4	1	混凝土	60	4	胜利路
5	1	混凝土	60	4	中山路
6	4	柏油	32	2	鲜花路

2.1 空间数据库模型概念

地理视图

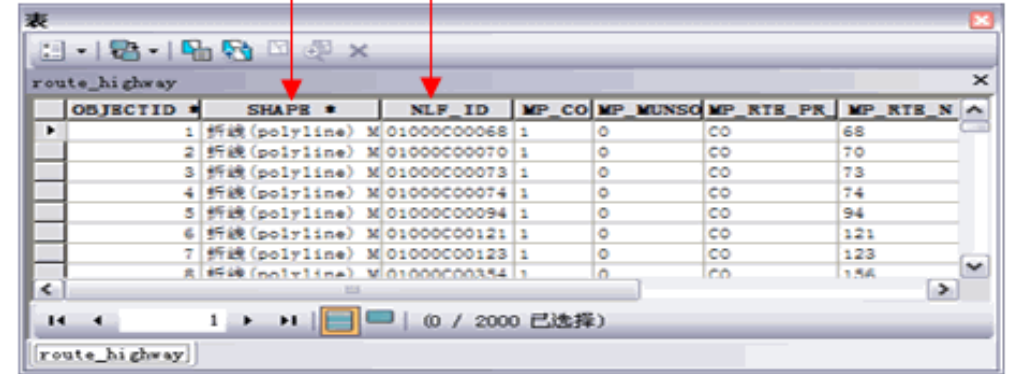


表视图

对象 ID	形状	名称	LV 编码	管理机构
1		成荫的针叶林	20	私有
2		松林村	30	松林村协会
3		Sarah 公园	80	城市公园委员会
4		城镇公园	99	城市公园委员会

具有测量值的
线状要素

唯一标识符



OBJECTID	SHAPE	NLF_ID	MP_CO	MP_MUNSC	MP_RTE_PR	MP_RTE_N
1	折线 (polyline)	M 01000C00068	1	0	CO	68
2	折线 (polyline)	M 01000C00070	1	0	CO	70
3	折线 (polyline)	M 01000C00073	1	0	CO	73
4	折线 (polyline)	M 01000C00074	1	0	CO	74
5	折线 (polyline)	M 01000C00094	1	0	CO	94
6	折线 (polyline)	M 01000C00121	1	0	CO	121
7	折线 (polyline)	M 01000C00123	1	0	CO	123
8	折线 (polyline)	M 01000C00354	1	0	CO	154

空间数据库中的矢量数据以要素类为单位进行组织，每个要素类在单独的表中进行管理，其中shape 列用于保存各要素的几何或形状。

在要素类表中，以下表述均为真：

- 每个要素类都是一张表。
- 各要素以行的形式进行存储。
- 要素属性以列的形式进行记录。
- shape 列保存各要素的几何（点、线和面等）。
- ObjectID 列保存各要素的唯一标识符。

2.1 空间数据库模型概念

空间数据库中的**属性数据**基于一系列简单且必要的关系数据概念在表中进行管理：

- 表包含行。
- 表中所有行具有相同的列。
- 每个列都有一个数据类型，例如，整型、十进制数字型、字符型和日期型。
- 可使用一系列关系函数和运算符（例如 SQL）在表及其数据元素上进行运算。

要素类表

Shape	ID	PIN	面积	地址	编码
	1	334-1626-001	7,342	341 Cherry Ct.	SFR
	2	334-1626-002	8,020	343 Cherry Ct.	UND
	3	334-1626-003	10,031	345 Cherry Ct.	SFR
	4	334-1626-004	9,254	347 Cherry Ct.	SFR
	5	334-1626-005	8,856	348 Cherry Ct.	UND
	6	334-1626-006	9,975	346 Cherry Ct.	SFR
	7	334-1626-007	8,230	344 Cherry Ct.	SFR
	8	334-1626-008	8,645	342 Cherry Ct.	SFR

Parcels feature class

Shape	ID	PIN	Area	Addr	Code
	1	334-1626-001	7,342	341 Cherry Ct.	SFR
	2	334-1626-002	8,020	343 Cherry Ct.	UND
	3	334-1626-003	10,031	345 Cherry Ct.	SFR
	4	334-1626-004	9,254	347 Cherry Ct.	SFR
	5	334-1626-005	8,856	348 Cherry Ct.	UND
	6	334-1626-006	9,975	346 Cherry Ct.	SFR
	7	334-1626-007	8,230	344 Cherry Ct.	SFR
	8	334-1626-008	8,645	342 Cherry Ct.	SFR

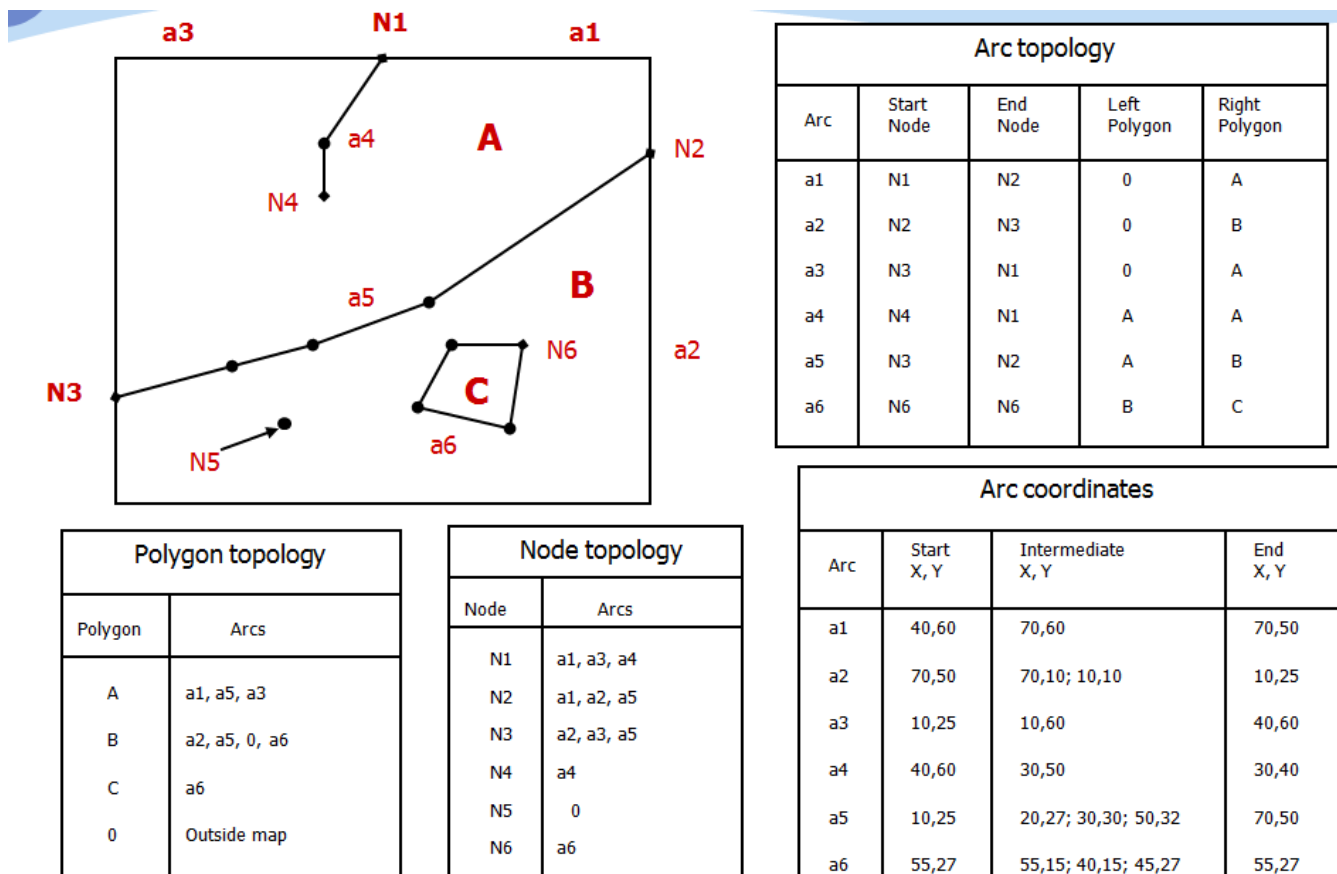
Related ownership table

PIN	Owner	Acq.Date	Assessed	TaxStat
334-1626-001	G. Hall	1995/10/20	\$115,500.00	02
334-1626-002	H. L. Holmes	1993/10/06	\$24,375.00	01
334-1626-003	W. Rodgers	1980/09/24	\$175,500.00	02
334-1626-004	J. Williamson	1974/09/20	\$135,750.00	02
334-1626-005	P. Goodman	1966/06/06	\$30,350.00	02
334-1626-006	K. Staley	1942/10/24	\$120,750.00	02
334-1626-007	J. Dormandy	1996/01/27	\$110,650.00	01
334-1626-008	S. Gooley	2000/05/31	\$145,750.00	02

2.1 空间数据库模型概念

采用数据库管理系统如何管理矢量数据的几何数据？

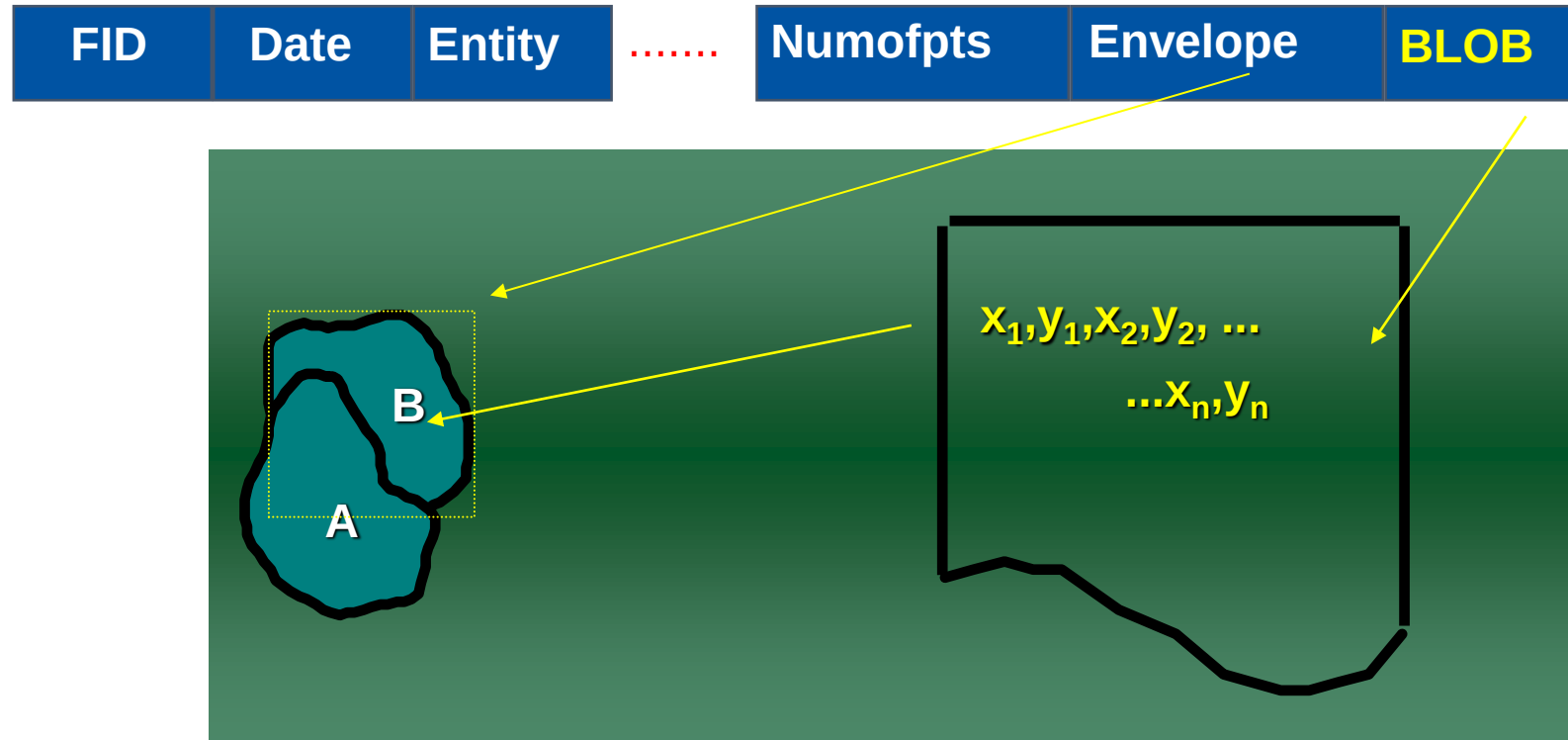
1、**基于关系模型的方式**:几何数据按关系数据模型组织。由于涉及一系列关系连接运算、费时。



2.1 空间数据库模型概念

采用数据库管理系统如何管理矢量数据的几何数据？

2、**将几何数据采用二进制字段（如BLOB）管理。**目前大部分关系数据库管理系统都提供了二进制块的字段域，以适应管理多媒体数据或可变长文本字符。



2.1 空间数据库模型概念

采用数据库管理系统如何管理矢量数据的几何数据？

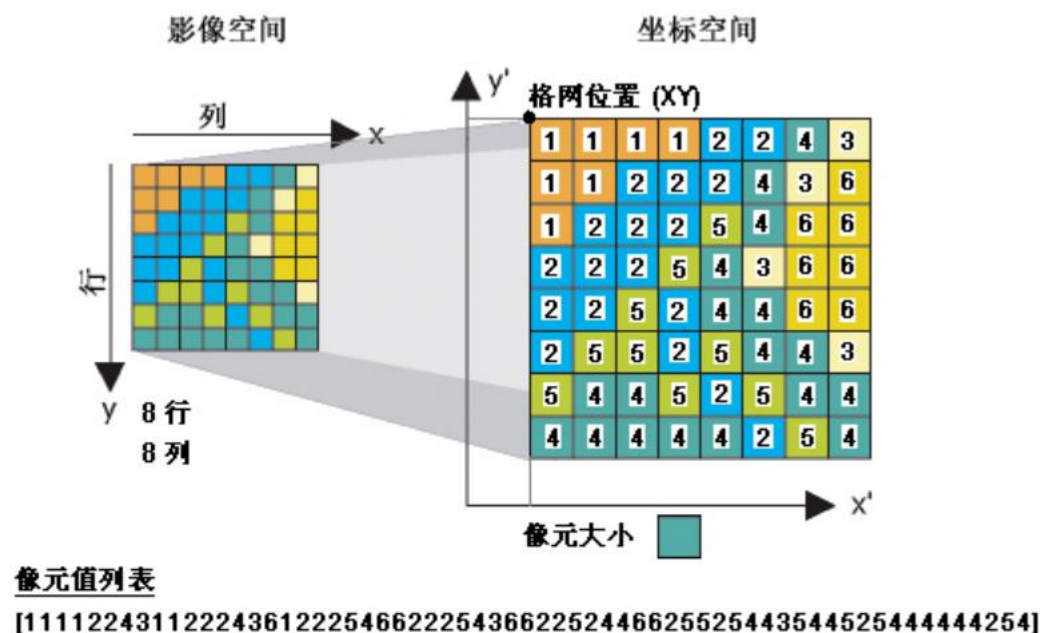
3、**数据库管理系统的软件商自己在关系数据库管理系统中进行扩展，使之能直接存储和管理矢量几何数据，如Oracle spatial 的SDO_GEOMETRY，Microsoft SQL Server的GEOMETRY 或GEOGRAPHY**

名称	数据类型	是否允许为空
NAME	VARCHAR2(32)*	
POPULATION	NUMBER(11)	
SHAPE	MDSYS.SDO_GEOMETRY	
OBJECTID	NUMBER(38)	非空

SDO_GEOMETRY	Oracle Spatial (包括 GeoRaster)
PG_GEOMETRY	PostGIS 几何类型
GEOMETRY	Microsoft 几何类型
GEOGRAPHY	Microsoft 地理类型

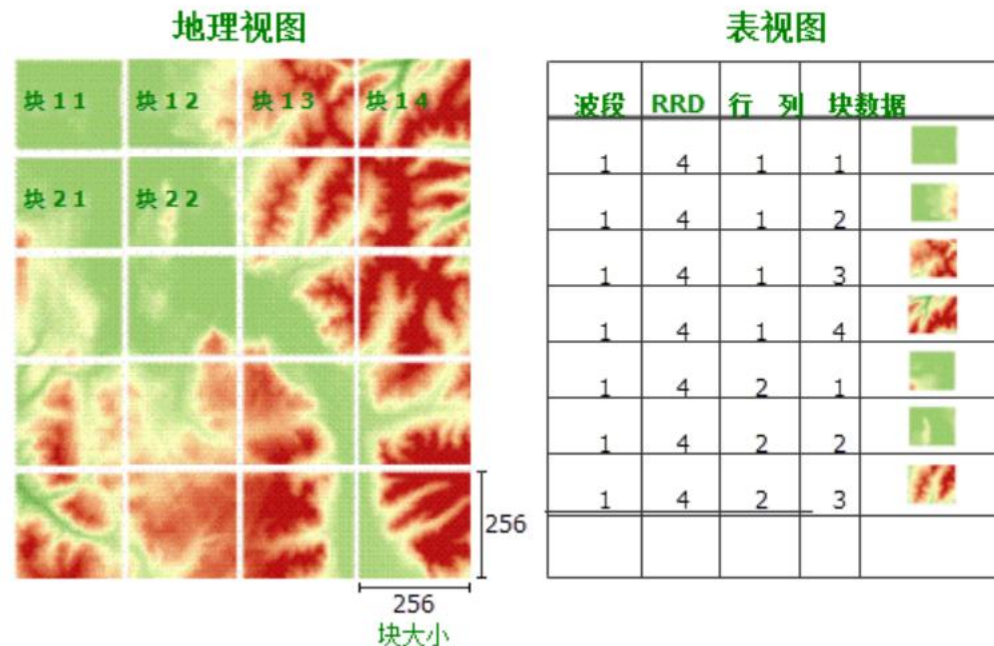
2.1 空间数据库模型概念

采用数据库管理系统如何管理栅格数据？



栅格的地理属性通常包括

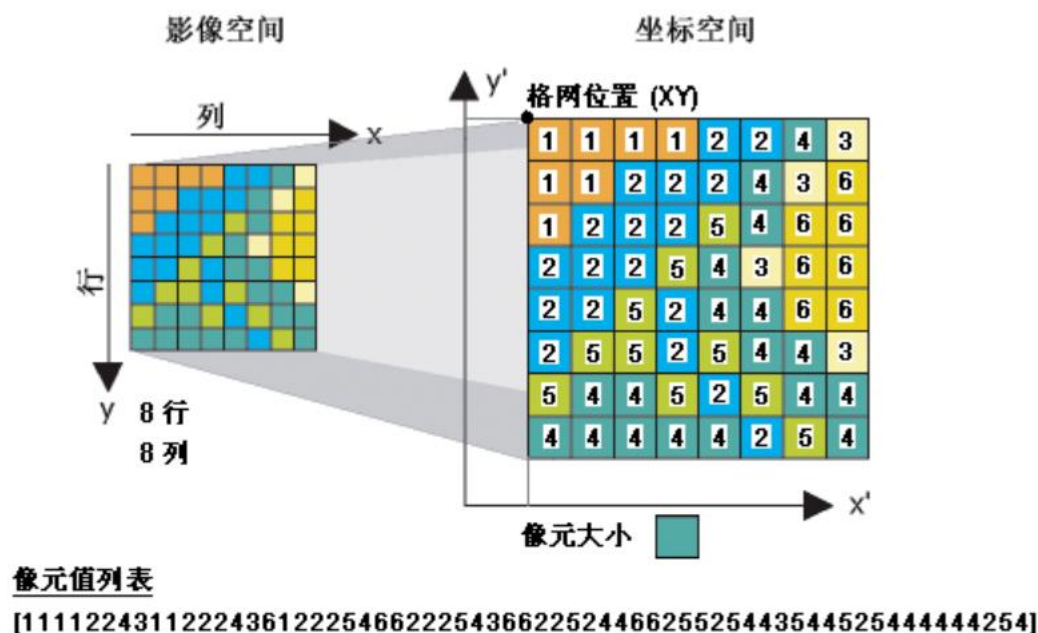
- 坐标系
- 参考坐标或 X, Y 位置 (通常在栅格左上角或左下角)
- 像元大小
- 行计数和列计数



为了获得高性能，可将空间数据库栅格分割成较小分块 (称为块)，其典型大小为大约 128 行与 128 列或 256 x 256。然后这些较小的块将保存在每个栅格的表中。每个单独的分块都保存在块表的单独的行中

2.1 空间数据库模型概念

采用数据库管理系统如何管理栅格数据?



DBMS	栅格列数据类型
DB2	BLOB
Informix	Byte
Oracle	BLOB LONG RAW
PostgreSQL	Bytea
SQL Server	Varbinary(Max) Image

栅格的地理属性通常包括

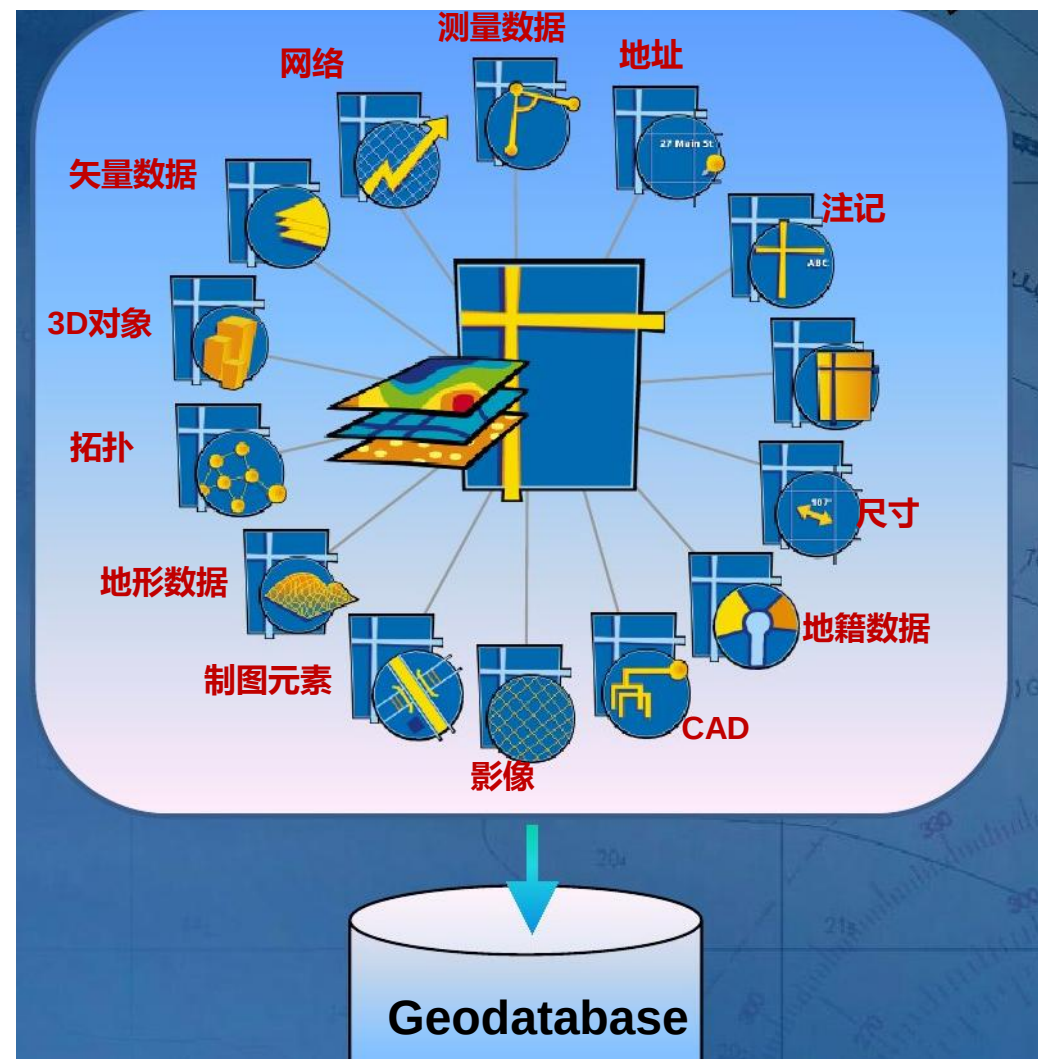
- 坐标系
- 参考坐标或 X,Y 位置 (通常在栅格左上角或左下角)
- 像元大小
- 行计数和列计数

2.2 Geodatabase

Geodatabase是一种基于关系数据库、采用面向对象技术来组织和管理空间数据的空间数据库模型。

Geodatabase中的数据对象就是逻辑数据模型中定义的对象(如建筑物、宗地和道路等)。

Geodatabase数据模型无需编写代码，通过ArcInfo提供的域、验证规则及其它功能可轻松实现大部分自定义行为 (仅建模特殊的要素行为时才需编写代码)。



2.2 Geodatabase

个人地理数据库(.mdb)

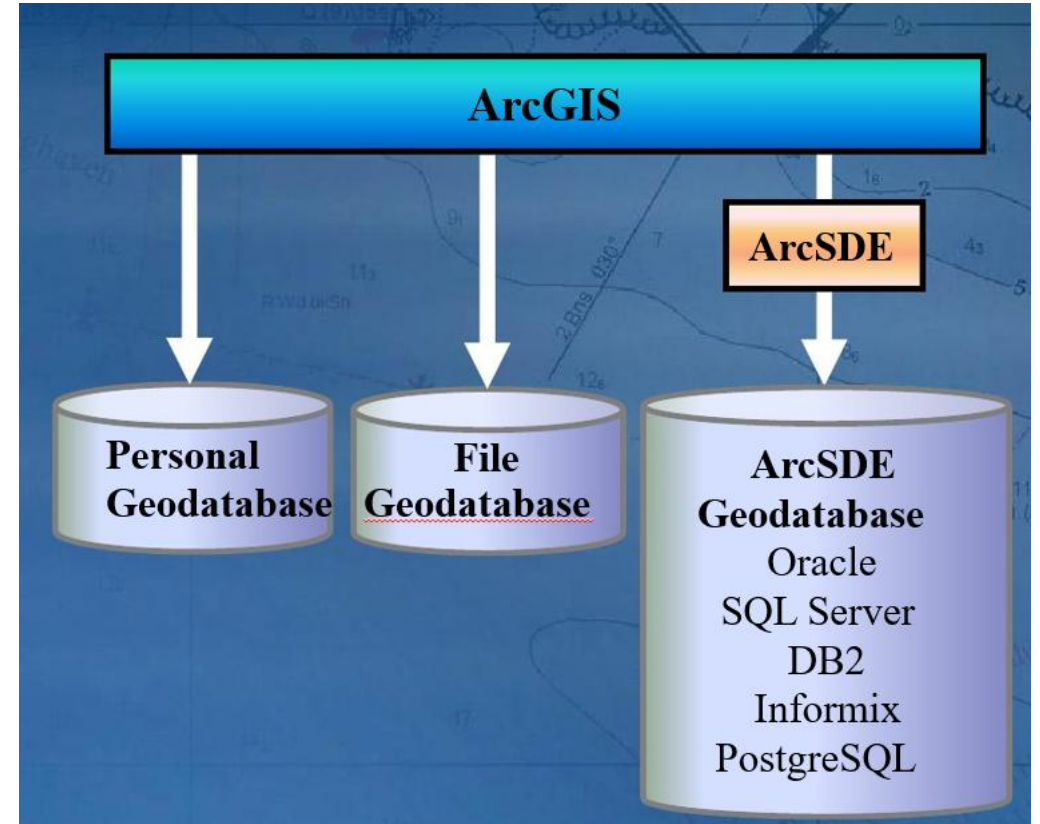
- FOR MS ACCESS
- 存储上限为2GB

文件地理数据库(.gdb)

- 在文件系统中以文件夹的形式表现
- 以二进制文件格式存储
- 每个表存储上限为1TB

ArcSDE地理数据库

- 支持多用户并发编辑
- 存储于RDBMS中
- 伸缩性



2.2 Geodatabase

Geodatabase是一种基于关系数据库、采用面向对象技术来组织和管理空间数据的空间数据库模型。

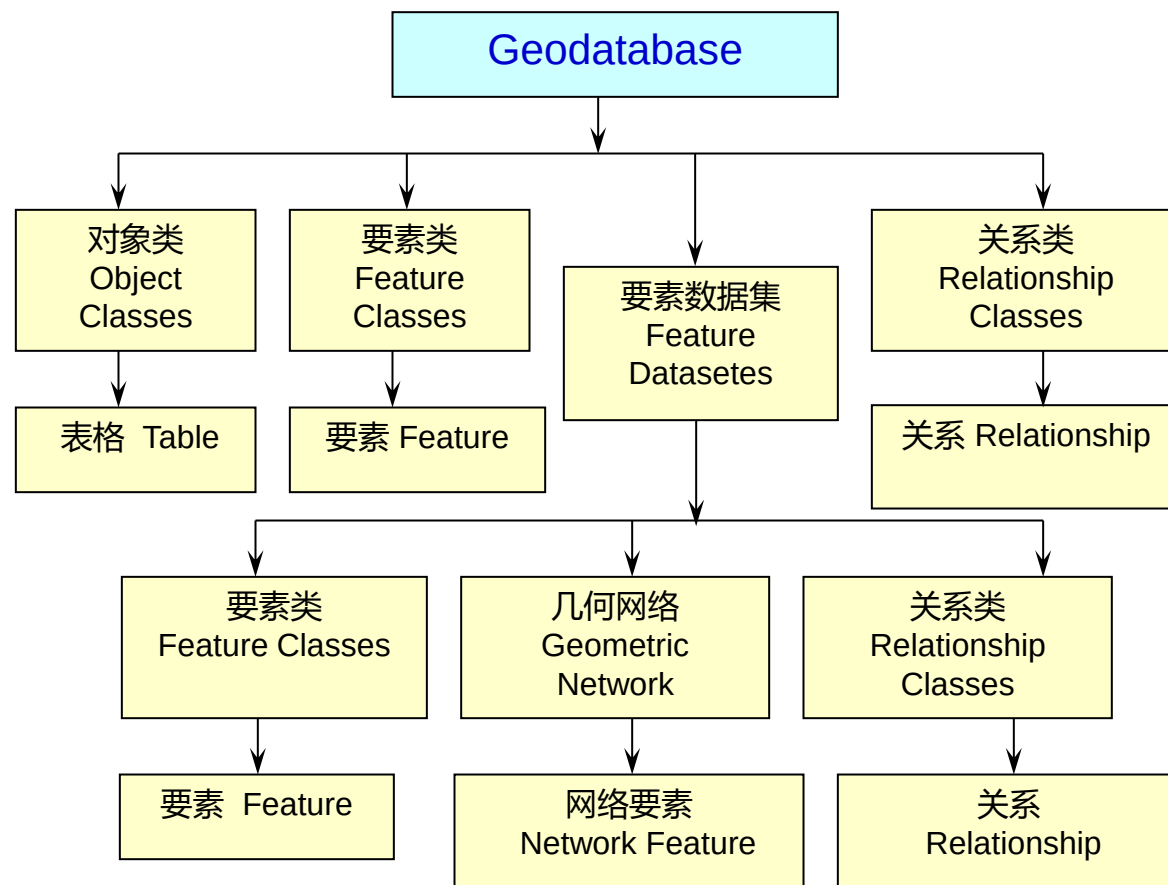
Geodatabase按照一定的模型和规则组合空间要素数据集(Feature Dataset)，它按层次型的数据对象(Object)来组织空间数据，这些数据对象包括：对象类、要素类、要素数据集和关系类。

对象类(Object Classes)：存储非空间数据的表(Table)；

要素类(Feature Classes)：具有相同几何类型和属性的要素的集合；

要素数据集(Feature Datasets)：共享空间参考系统的要素类的集合；

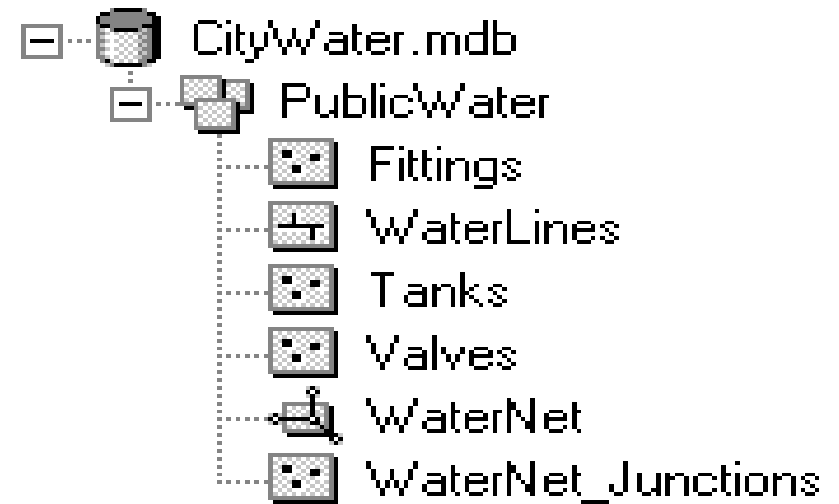
关系类(Relationship Classes)：存储两个对象类或要素类中的实体间的关联关系的表。



2.2 Geodatabase

Geodatabase中通过三种空间数据集实现三种空间数据模型

- **要素数据集** (Feature dataset)
 - 是具有同一空间参考的要素类的群集。
- **栅格数据集** (Raster dataset)
 - 可以是简单数据集，也可以是具有不同光谱值或分类值的多波段复合数据集
- **TIN数据集** (TIN dataset)
 - 包含一组三角形，这些三角形通过精确地覆盖某个区域来表示某种表面，三角形的每个顶点具有z值。



在 CityWater geodatabase中, 三个点要素类和一个线要素类被组合为PublicWater 要素集并产生几何网络WaterNet

2.2 Geodatabase

要素数据集 (Feature dataset)

◆ 存储空间数据的容器

相同的空间参考

外部要素类导入时自动投影转换

不能存储表格

◆ 支持多种行为

- 拓扑
- 几何网络
- 网络数据集
- 地形
- 关系类



2.2 Geodatabase

栅格数据集 (Raster dataset)

◆ 以波段形式组织的有效栅格数据格式

TIFF、BMP、GRIDS

单波段或多波段

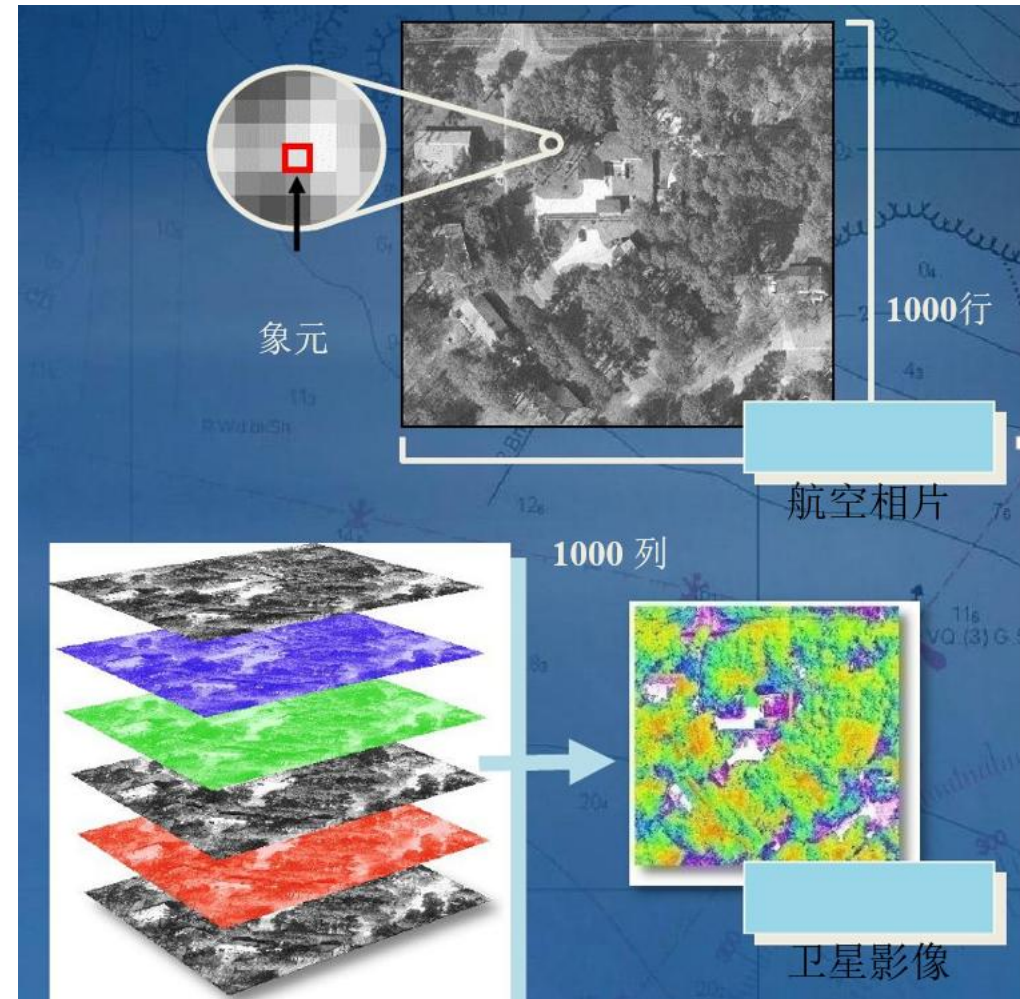
波段 ~ 象元矩阵

◆ 数据来源

- 卫星影像、航空相片、扫描图片

◆ 栅格数据集的构成

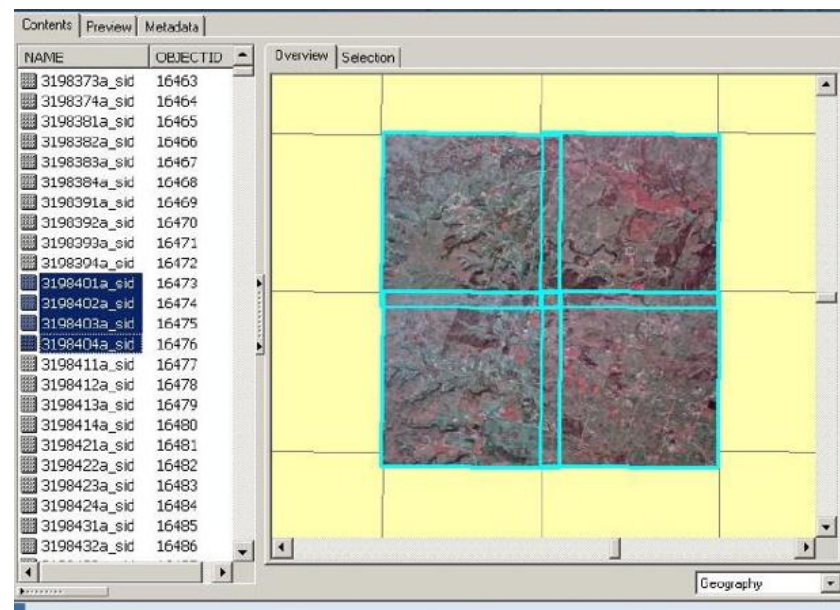
- 独立的栅格数据
- 加载时可镶嵌
 - 相同分辨率



2.2 Geodatabase

栅格目录

- ◆ 栅格数据集的集合
- ◆ 采用表格的形式
 - 一条记录对应一个栅格数据集
- ◆ 适用情况
 - 范围部分或完全重叠，想保留公共区域
 - 完全重叠，且作为某一时 范围间序列的一部分
 - 不需要一次显示全部区域
 - 需要保留元数据



2.2 Geodatabase

要素类(Feature Classes)

要素类(Feature class)是具有相同的几何图形类型(点/线/多边形)的空间要素的群集。

- 简单要素类**(Sample Feature Classes)

包含点、线、多边形和注记, 且它们之间没有任何拓扑关系。

例如: 一个要素类中的点与另一要素类中线的终点可能同时存在, 但它们是不同的点, 这两个点要素可以独立编辑。

- 拓扑要素类**(Topological Feature Classes)

拓扑要素类被限定在一幅图(graph)中。图是一个对象, 它把组成有机拓扑单元(**节点Junction**、**边Edge**)的一组要素类捆绑起来。

Feature (row)	PARCEL <i>FeatureClass (table)</i>			
	OID	Shape	Type	...
	524	X,Y,Z,M, ...	Private	...

2.2 Geodatabase

关系类(Relationship Classes)

- 定义两个不同的要素类或对象类之间的关联关系。例如：房主和房子之间的关系，房子和地块之间的关系等
- 在 Geodatabase 中，关系类提供了一种对现实世界中对象间相互关系建模的方法。



地块(源类)			建筑物 (目的类)		
Parcel_ID	Zone	Block	Building_ID	Parcel_ID	Floors
...	...	...	...	...	...
123	...	...	...	123	...
...	...	...	...	123	...
...	...	...	...	...	...

简单关系：对象间相对独立

地块(源类)			建筑物 (目的类)		
Parcel_ID	Zone	Block	Building_ID	Parcel_ID	Floors
...	...	...	...	...	...
123	...	...	...	123	...
...	...	...	...	123	...
...	...	...	...	...	...

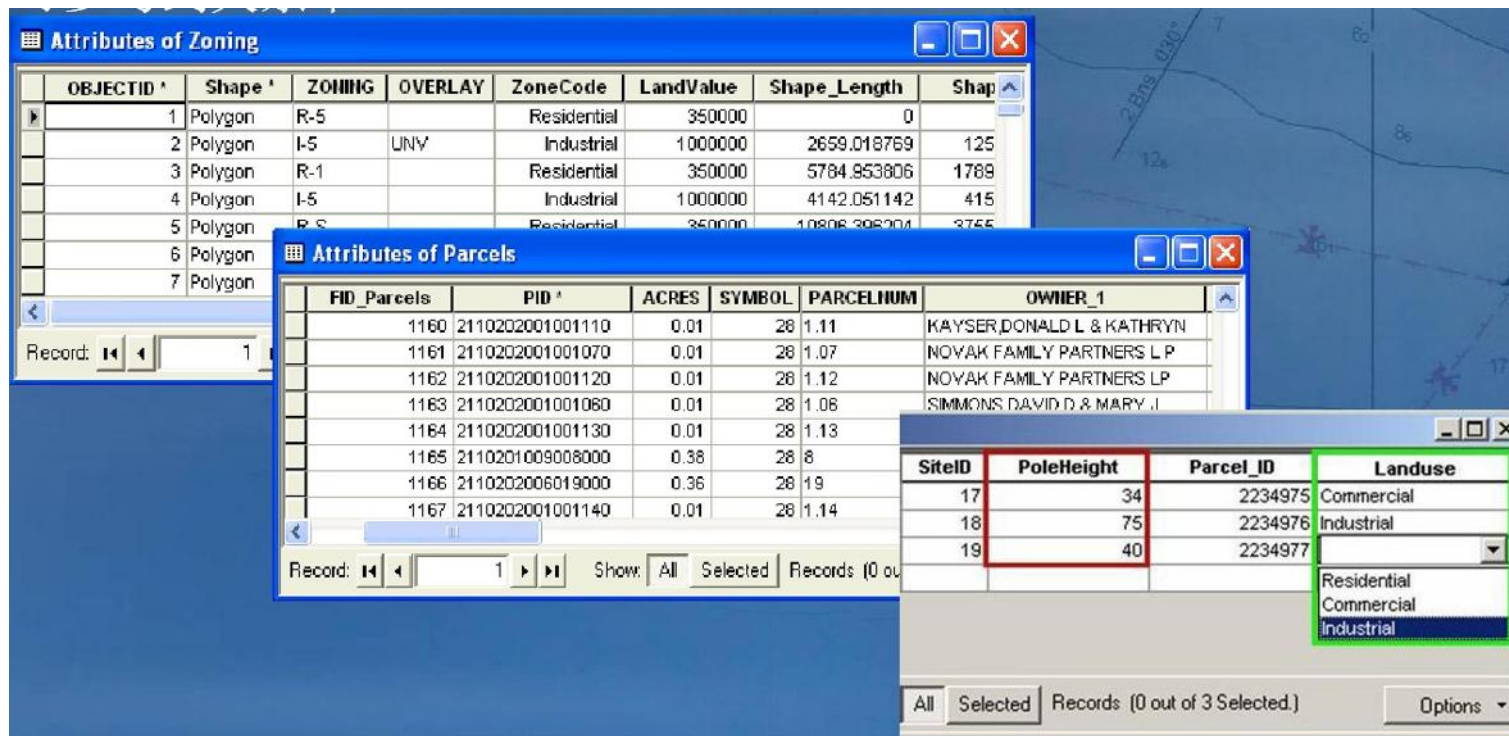
复合关系：对象间可传递消息，相互影响

2.2 Geodatabase

对象类(Object Class)

- 对象类(Object class)是Geodatabase中的一个表(Table)，它保存了与空间要素相关的对象的描述性信息，它不等同于地图上的要素。

- 行列的集合，具有属性和行为
 - 可存储属性值、地址、坐标对、路径事件
 - 可定义行为：子类型、属性域、缺省值
- 可参与到关系中



OBJECTID	Shape	ZONING	OVERLAY	ZoneCode	LandValue	Shape_Length	Shap
1	Polygon	R-5		Residential	350000	0	
2	Polygon	I-5	UNV	Industrial	1000000	2659.018769	125
3	Polygon	R-1		Residential	350000	5784.953806	1789
4	Polygon	I-5		Industrial	1000000	4142.051142	415
5	Polygon	R-5		Residential	350000	10806.308204	3755
6	Polygon						
7	Polygon						

FID_Parcels	PID	ACRES	SYMBOL	PARCELNUM	OWNER_1
1160	2110202001001110	0.01	28	1.11	KAYSER, DONALD L & KATHRYN
1161	2110202001001070	0.01	28	1.07	NOVAK FAMILY PARTNERS L P
1162	2110202001001120	0.01	28	1.12	NOVAK FAMILY PARTNERS LP
1163	2110202001001080	0.01	28	1.08	SIMMONS, DAVID D & MARY J
1164	2110202001001130	0.01	28	1.13	
1165	2110201009008000	0.38	28	8	
1166	2110202006019000	0.36	28	19	
1167	2110202001001140	0.01	28	1.14	

SiteID	PoleHeight	Parcel_ID	Landuse
17	34	2234975	Commercial
18	75	2234976	Industrial
19	40	2234977	Industrial

2.2 Geodatabase

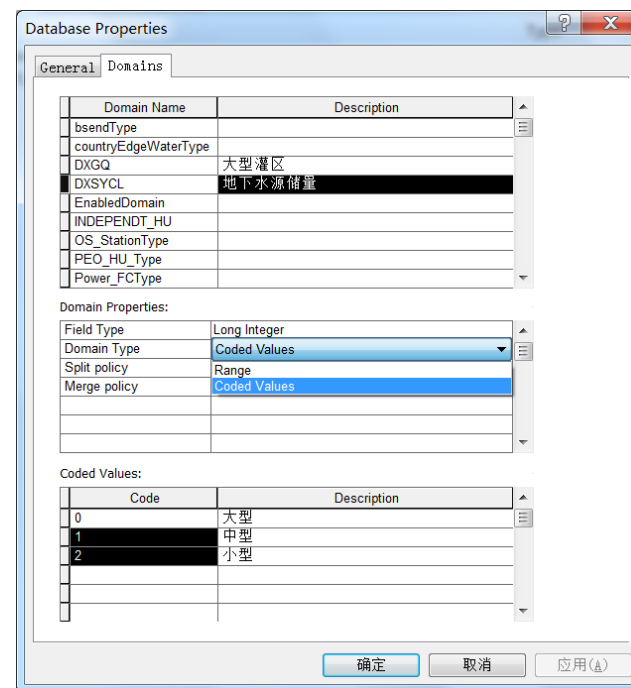
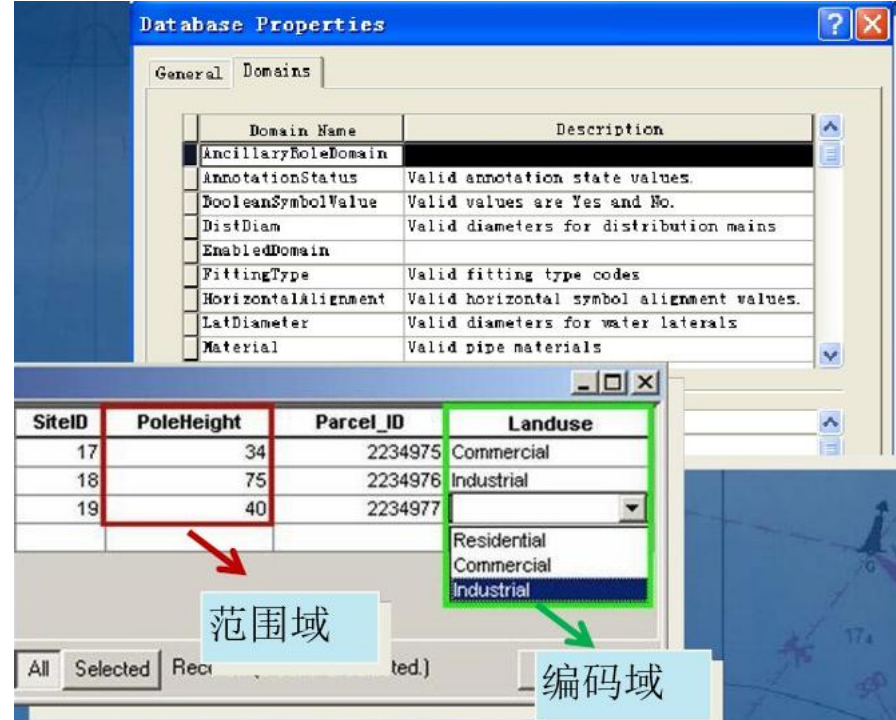
Geodatabase中的行为

◆ 属性域 (Domains)

- 定义一个属性字段的合法属性值
- 在Geodatabase级别上定义
- 两种类型
范围属性域 (Range)
编码值属性域(Coded Value)

■ Range：取值区间。如，10000~50000立方米/秒。

■ Code：枚举。如，大型水库、中型水库、小型水库。



2.2 Geodatabase

Geodatabase中的行为

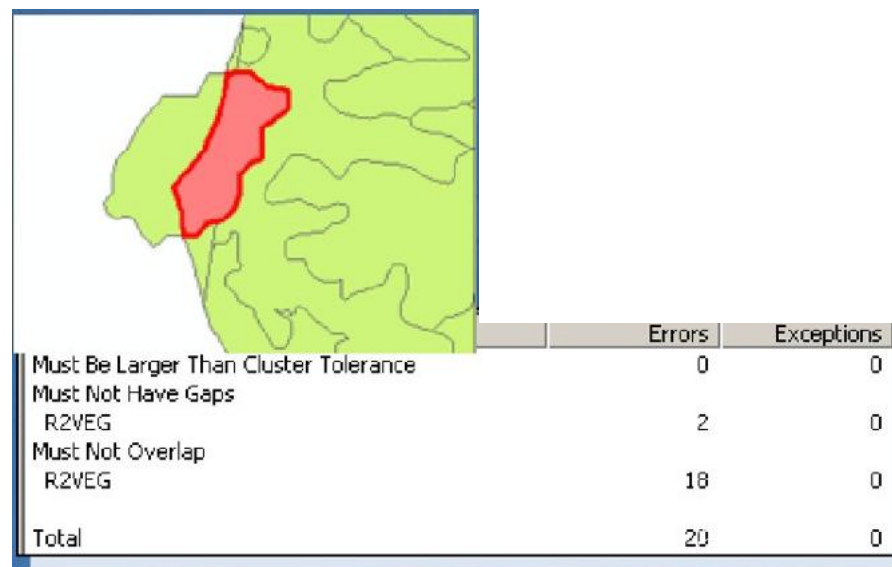
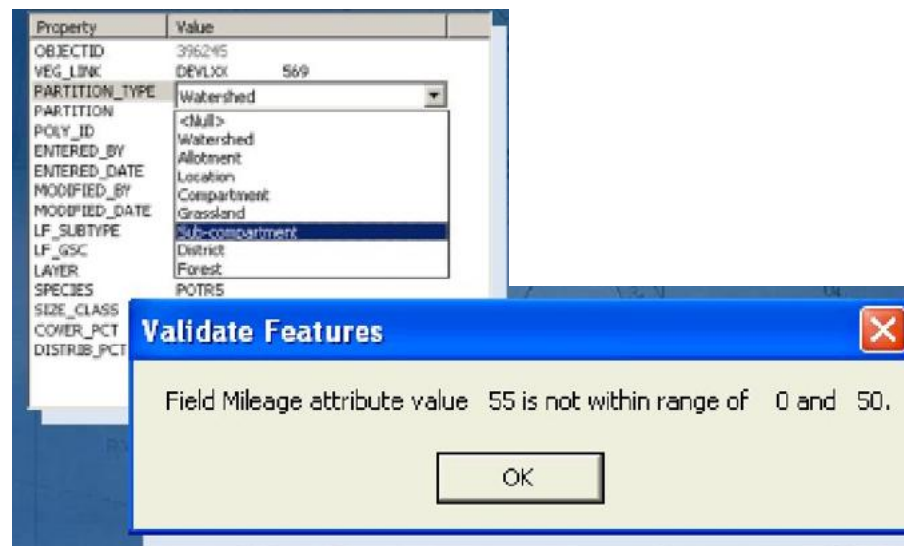
◆ 有效性规则

• Geodatabase中包含:

- 属性规则
- 连通性规则
- 关系规则
- 拓扑规则

• 使用规则可以:

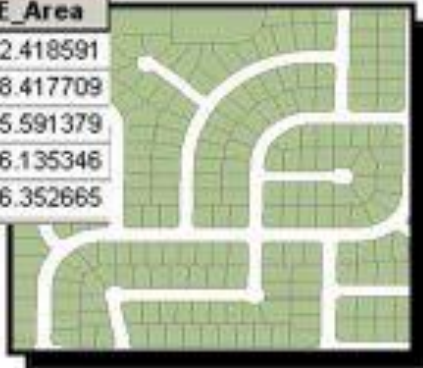
- 方便的管理分组要素
- 提供一个有效值列表
- 定位属性错误
- 属性缺省值
- 定位空间错误
- 保证关系类的有效性和几何网络的连通性
- 实现高效的编辑



2.2 Geodatabase

- Geodatabase体系结构基于一系列简单，但是非常重要的数据库概念之上，即DBMS：
 - 数据存放在表中；
 - 表包含了记录；
 - 所有表中记录包含了相同的列；
 - 每个列都有数据类型，例如Integer，Decimal number，Character，Date等；
 - 关系用于关联一个表的记录与另外一个表的记录，一般通过表中相同的列来进行，这两个列被称为主键和外键。

Parcels							
	OBJECTID *	SHAPE *	PROPERTY_I *	Res	Zoning_simple	SHAPE_Length	SHAPE_Area
	1	面	5001	Non-Residential	<空>	3597.780813	112552.418591
	2	面	5002	Non-Residential	<空>	814.855837	18488.417709
	3	面	1003	Residential	Residential	489.655523	12815.591379
	4	面	1004	Residential	Residential	521.761248	14036.135346
	5	面	1005	Residential	Residential	453.479649	9816.352665



2.2 Geodatabase

➤空间数据一般存储为矢量要素和栅格数据，以及传统意义上属性表。比如：一个DBMS表可以用来存放一个要素的集合，表中的每行可以用来保存一个要素。每行中的shape字段存储要素的空间几何或形状信息。**shape字段的类型一般分为两种：BLOB、DBMS支持的空间类型**

对于 SQL Server 数据库，可以使用 ArcSDE 压缩二进制（默认）存储方法、Open Geospatial Consortium, Inc. (OGC) 熟知二进制 (WKB) 存储方法，或者 Microsoft 的几何或地理空间类型。

对于 Oracle 数据库，可使用 ArcSDE 压缩二进制、OGC WKB、ST_Geometry 或 Oracle Spatial。对于 PostgreSQL，可使用 ST_Geometry 或 PostGIS 几何类型。

配置关键字	几何存储
WKB_GEOMETRY	OGC 熟知二进制类型 (WKB)
SDELOB	存储为二进制大对象 (BLOB) 的 ArcSDE 压缩二进制
SDEBINARY	ArcSDE 压缩二进制
ST_GEOMETRY	Oracle 或 PostgreSQL 的空间类型
SDO_GEOMETRY	Oracle Spatial (包括 GeoRaster)
PG_GEOMETRY	PostGIS 几何类型
GEOMETRY	Microsoft 几何类型
GEOGRAPHY	Microsoft 地理类型

2.2 Geodatabase

- **基于表的数据集具有相关的完整性规则。**例如，每个记录具有相同的列，而域列出该列合法的值的集合或范围。通过域和验证规则， Geodatabase 能够增强属性的完整性。
- **具有一系列函数和操作符来对表和数据进行操作。**
- **能够将要素的自然行为绑定到存储要素的表中。**
- **能够展现多个版本，以便多个用户编辑同一份数据**

目录

CONTENTS

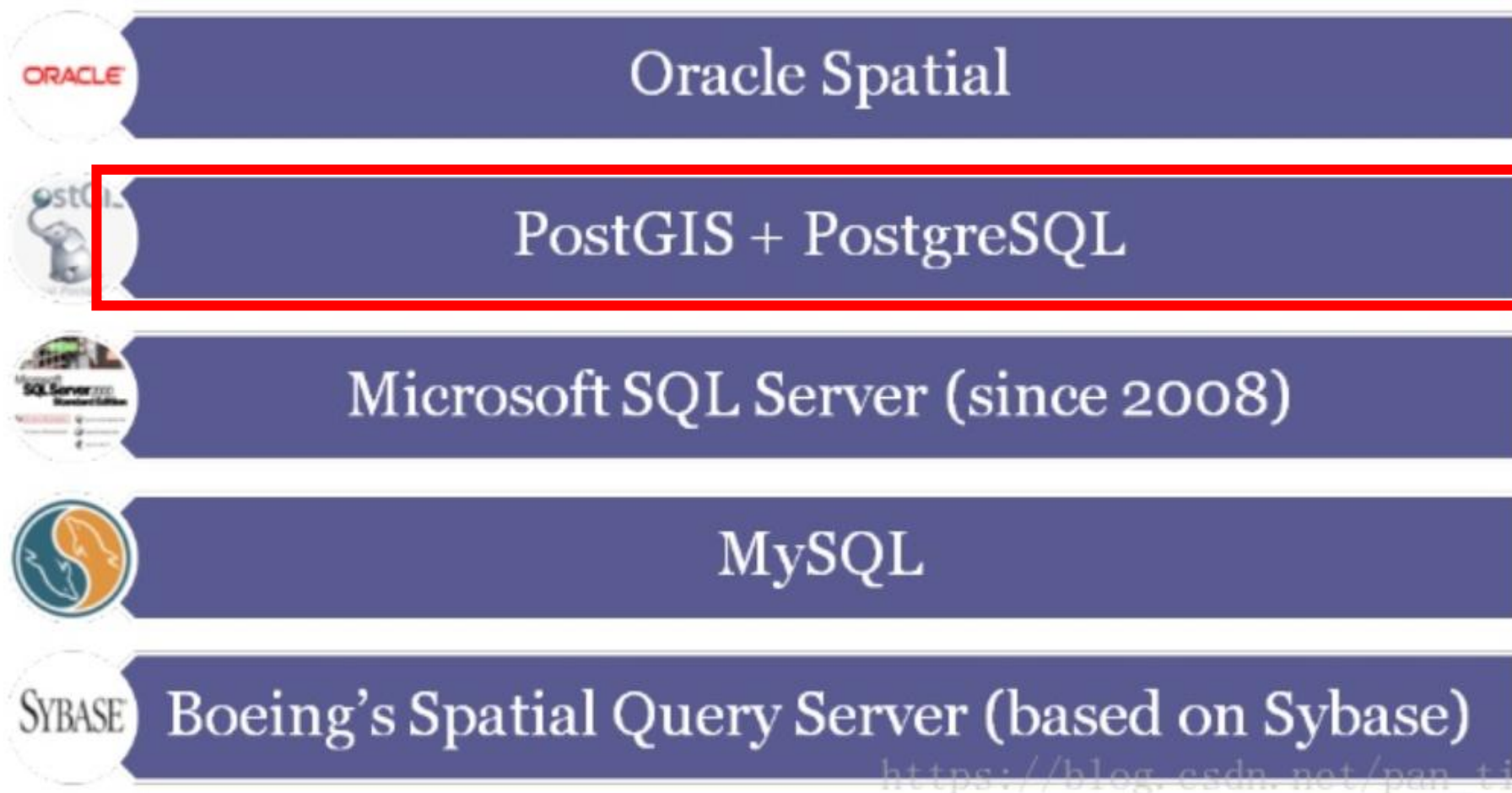
3

集成式的空间数据库模型

3.1 关系数据库管理系统的空间扩展

3.2 PostGIS概述

3.1 关系数据库管理系统的空间扩展



PostGIS概述

- PostGIS是对象关系型数据库系统PostgreSQL的一个扩展。
- PostGIS提供如下空间信息服务功能:空间对象、空间索引、空间操作函数和空间操作符。
- PostGIS遵循OpenGIS的规范。
 - 基于空间对象库GEOS和空间投影库PROJ.4开发
 - 支持桌面GIS软件: GRASS, QGIS, uDig, JUMP
 - 支持中间件服务器: MapServer, GeoServer等
 - 支持开发库: GeoTools, OGR
 - ESRI ArcGIS 9.3开始支持PostGIS空间数据类型



GEOS: 经典的空间函数库， C++ 写的， 提供了各种拓扑运算的核心功能。

<http://trac.osgeo.org/geos>

拓扑关系判断：相等(Equals)、脱节(Disjoint)、相交(Intersects)、接触(Touches)、交叉(Crosses)、内含(Within)、包含(Contains)、重叠(Overlaps)等

拓扑操作：联合(Union)、距离计算(Distance)、交叉分析(Intersection)、差异(Difference)、对等差分 (SymDifference)、凸包分析(Convex Hull)、缓冲区分析(Buffer)、最小外接矩形(Envelope)、抽稀 (Simplify) 等



PROJ: Proj.4经典的空间坐标转换库，用于各种地理坐标和投影坐标转换。

<https://proj4.org/>

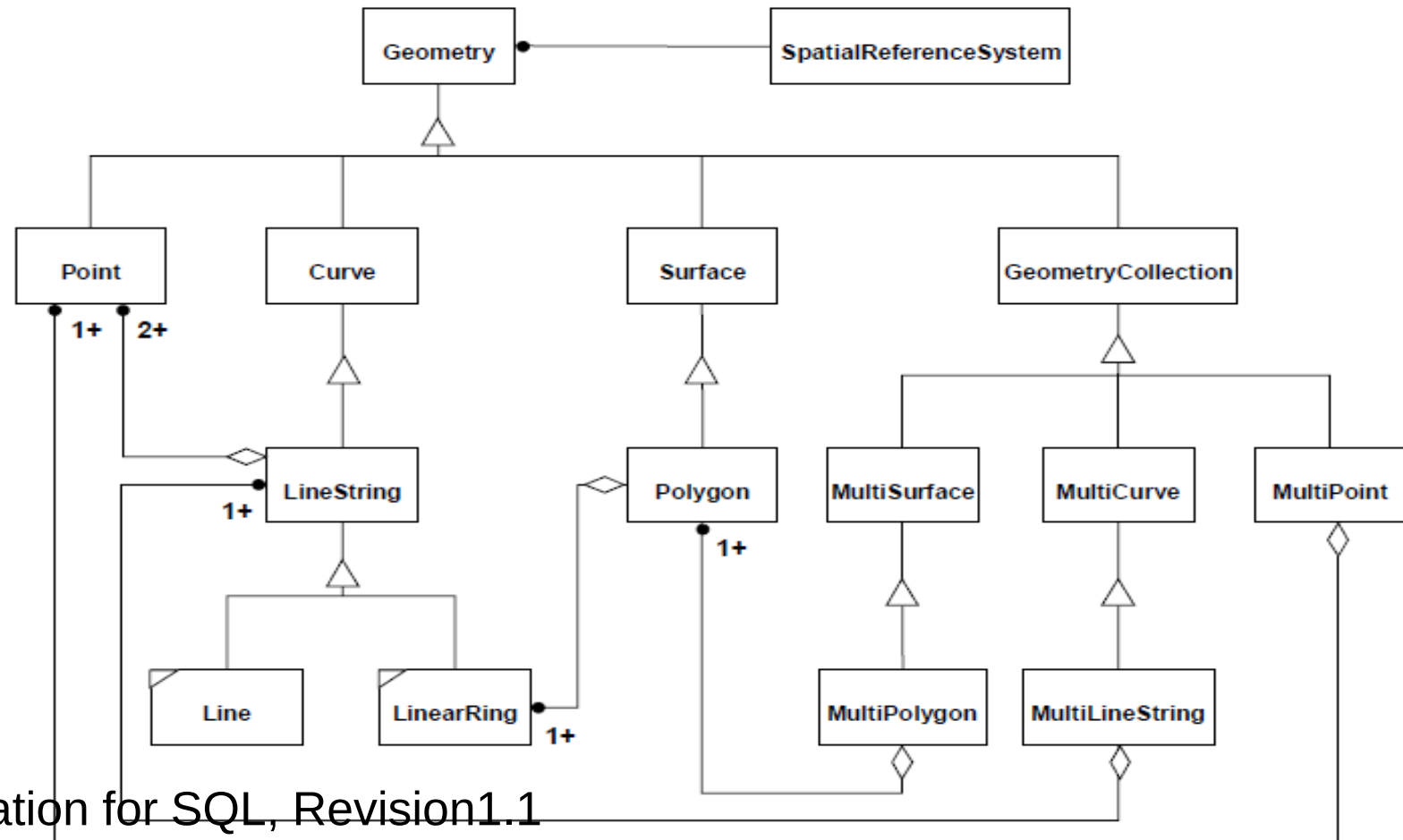
内置了超多坐标系（EPSG的+ESRI的），支持不同坐标系坐标随意转换

也有一个java版的，叫Proj4J <https://trac.osgeo.org/proj4j/>

PostGIS支持的空间类型

• 支持OpenGIS中所有空间数据类型

- POINT, LINESTRING, POLYGON
- MULTI-POINT, MULTI-LINESTRING, MULTI-POLYGON,
- GEOMETRY COLLECTION



PostGIS支持的空间类型

Spatial Data Type WKT as defined by the OGC

- Examples of WKT:
 - POINT(0 0)
 - LINESTRING(0 0,1 1,1 2)
 - POLYGON((0 0,4 0,4 4,0 4,0 0),(1 1, 2 1, 2 2, 1 2,1 1))
 - MULTIPOINT(0 0,1 2)
 - MULTILINESTRING((0 0,1 1,1 2),(2 3,3 2,5 4))
 - MULTIPOLYGON((((0 0,4 0,4 4,0 4,0 0),(1 1,2 1,2 2,1 2,1 1)), ((-1 -1,-1 -2,-2 -2,-2 -1,-1 -1)))
 - GEOMETRYCOLLECTION(POINT(2 3),LINESTRING(2 3,3 4))

■ PostGIS支持的空间类型

- 除了OpenGIS定义的地理数据类型之外，PostGIS还对数据类型进行了扩展，这种扩展主要是两方面的扩展：
- 一是把**二维的数据向三维和四维扩展**；
- 二是在**WKT和WKB数据类型基础上扩展出EWKT和EWKB数据类型**。
 - EWKT, EWKB（包含了SRID信息的WKT/WKB）
 - SRID(Spatial Referencing System Identifier)：每个空间实例都有一个空间引用标识符(SRID)。SRID 对应于基于特定椭圆体的空间引用系统，可用于平面球体映射或圆球映射。空间列可包含具有不同 SRID 的对象。

PostGIS读写数据

- 使用psql语言

- Psql语言是PostgreSQL内嵌的一个命令行工具，其语法基本上和标准的SQL语法是一致的，可以使用Psql工具，结合标准SQL语法和一些PostGIS的扩展对PostGIS数据库进行读写操作。
- 这种方式功能强大，但全部需要手工操作。

```
select * from test1;  
select myID,AsText(pt) from test1;  
select Distance(pt, 'POINT(0 0)') from test1;
```

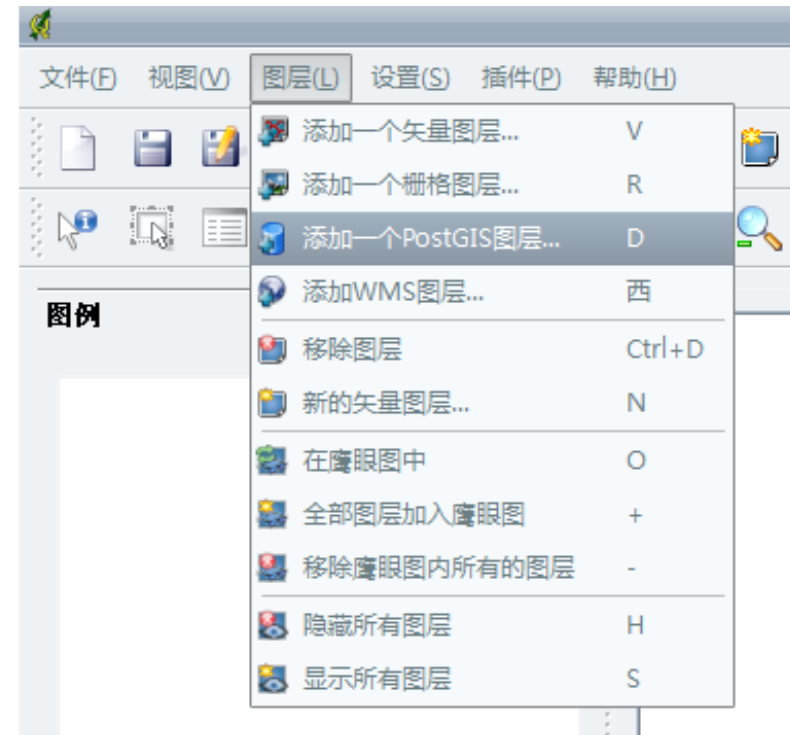
PostGIS读写数据

- 有两个很有用的小的转换工具，一是shp2pg; 一是ogr2ogr。
- shp2pgsql和pgsql2shp
 - shp2pgsql和pgsql2shp是PostGIS自身携带的一对在Shape文件和PostGIS数据库之间进行转换的工具。
 - 使用范围有限，只针对Shape文件格式
- ogr2ogr
 - ogr是转换矢量GIS数据的软件库。
 - 目前ogr能够支持的数据格式包括：
 - Arc/Info Binary Coverage、DWG、ESRI Personal GeoDatabase、ArcSDE、ESRI Shapefile、GML、GRASS、Mapinfo File、Microstation DGN、ODBC、Oracle Spatial和PostgreSQL等。应该说，这就基本包括了我们平常用到的所有矢量型GIS文件格式了。

PostGIS读写数据

• 在其他GIS软件中读写PostGIS数据

- 比如在QGIS中，能够打开PostGIS图层，还有SPIT插件可以把Shape文件输入到PostGIS数据库中。
- 其他GIS软件如uDig, Grass等，甚至连ArcGIS都支持或部分支持读写PostGIS数据。



- **利用接口在应用程序中读写PostGIS数据**

- 广大的开源GIS程序员几乎为每一种程序设计语言设计好了读写PostGIS的接口，如利用PostgreSQL的JDBC库，可以使用Java语言在程序中读写PostGIS数据；利用libpq库，可以使用C语言读写PostGIS数据。

- 表的建立

- **Create a normal non-spatial table.**

```
CREATE TABLE ROADS_GEOM ( ID int4, NAME varchar(25) )
```

- **Add a spatial column to the table using the OpenGIS "AddGeometryColumn" function.**

```
AddGeometryColumn(  
  <schema_name>,  
  <table_name>,  
  <column_name>,  
  <srid>,  
  <type>,  
  <dimension>  
)
```

Or

```
AddGeometryColumn(  
  <table_name>,  
  <column_name>,  
  <srid>,  
  <type>,  
  <dimension>  
)
```

- 表的建立

```
CREATE TABLE parks (  
  park_id INTEGER,  
  park_name VARCHAR,  
  park_date DATE,  
  park_type VARCHAR  
);  
SELECT AddGeometryColumn('parks', 'park_geom', 128, 'MULTIPOLYGON',  
2 );
```

Example1:

```
SELECT AddGeometryColumn('public', 'roads_geom', 'geom',  
423, 'LINESTRING', 2)
```

Example2:

```
SELECT AddGeometryColumn( 'roads_geom', 'geom', 423, 'LINESTRING', 2)
```

PostGIS查询语言

• 表中插入数据

```
BEGIN;
INSERT INTO roads (road_id, roads_geom, road_name)
VALUES (1, ST_GeomFromText('LINESTRING(191232 243118,191108 243242)',-1),'Jeff Rd');
INSERT INTO roads (road_id, roads_geom, road_name)
VALUES (2, ST_GeomFromText('LINESTRING(189141 244158,189265 244817)',-1),'Geordie Rd');
INSERT INTO roads (road_id, roads_geom, road_name)
VALUES (3, ST_GeomFromText('LINESTRING(192783 228138,192612 229814)',-1),'Paul St');
INSERT INTO roads (road_id, roads_geom, road_name)
VALUES (4, ST_GeomFromText('LINESTRING(189412 252431,189631 259122)',-1),'Graeme Ave');
INSERT INTO roads (road_id, roads_geom, road_name)
VALUES (5, ST_GeomFromText('LINESTRING(190131 224148,190871 228134)',-1),'Phil Tce');
INSERT INTO roads (road_id, roads_geom, road_name)
VALUES (6, ST_GeomFromText('LINESTRING(198231 263418,198213 268322)',-1),'Dave Cres');
COMMIT;
```

Edit Data - PostgreSQL 8.3 (localhost:5432) - template_postgis - geometry_columns									
文件(F) 编辑(E) 视图(V) 帮助(H)									
不限制									
	oid	f_table_cat [PK] character	f_table_sch [PK] character	f_table_name [PK] character	f_geometry_type [PK] character	coord_dimension integer	srid integer	type character va	
1	17324	"	public	gline1	geomy	2	-1	LINESTRING	
2	17576	"	public	glinetest	geom	2	-1	LINESTRING	
3	17219	"	public	gtest	geom	2	-1	LINESTRING	
4	17334	"	public	linesegment	geom	2	-1	LINESTRING	
5	17466	"	public	smallshop	geom	2	-1	POINT	
6	17366	"	public	smallshops	geom	2	-1	POINT	
*									

- 查询表数据

```
db=# SELECT road_id, ST_AsText(road_geom) AS geom, road_name FROM roads;
```

road_id	geom	road_name
1	LINESTRING(191232 243118,191108 243242)	Jeff Rd
2	LINESTRING(189141 244158,189265 244817)	Geordie Rd
3	LINESTRING(192783 228138,192612 229814)	Paul St
4	LINESTRING(189412 252431,189631 259122)	Graeme Ave
5	LINESTRING(190131 224148,190871 228134)	Phil Tce
6	LINESTRING(198231 263418,198213 268322)	Dave Cres
7	LINESTRING(218421 284121,224123 241231)	Chris Way

(6 rows)

PostGIS查询语言

- 空间查询例子:

- 1. What is the total length of all roads, expressed in kilometers?

```
SELECT sum(ST_Length(the_geom))/1000 AS  
km_roads FROM bc_roads;  
km_roads  
-----  
70842.1243039643  
(1 row)
```

PostGIS查询语言

- 空间查询例子:

- 2. How large is the city of Prince George, in hectares?

```
SELECT  
ST_Area(the_geom)/10000 AS hectares  
FROM bc_municipality  
WHERE name = 'PRINCE GEORGE';  
hectares  
-----  
32657.9103824927  
(1 row)
```




- **PostGIS函数大致可以分为以下四类：**

- **字段处理函数**

- AddGeometryColumn为已有的数据表增加一个地理几何数据字段；
- DropGeometryColumn删除一个地理数据字段的；
- SetSRID设置SRID值

- **几何关系函数**

- 这类函数目前共有10个，分别是：
- Distance, Equals, Disjoint, Intersects, Touches Crosses, Within, Overlaps, Contains, Relate



- **PostGIS函数大致可以分为以下四类：**

- **几何分析函数**

- 这类函数目前共有12个，分别是：
- Centroid, Area, Lenth, PointOnSurface, Boundary, Buffer, ConvexHull, Intersection, SymDifference, Difference, GeomUnion, MemGeomUnion

- **读写函数**

- 这类函数很多，主要是用于在各种数据类型之间的转换，尤其是在于Geometry数据类型与其他如字符型等数据类型之间的转换，函数名如AsText、GeomFromText等。

谢谢大家!

